



HANDS ON INTERVENTION

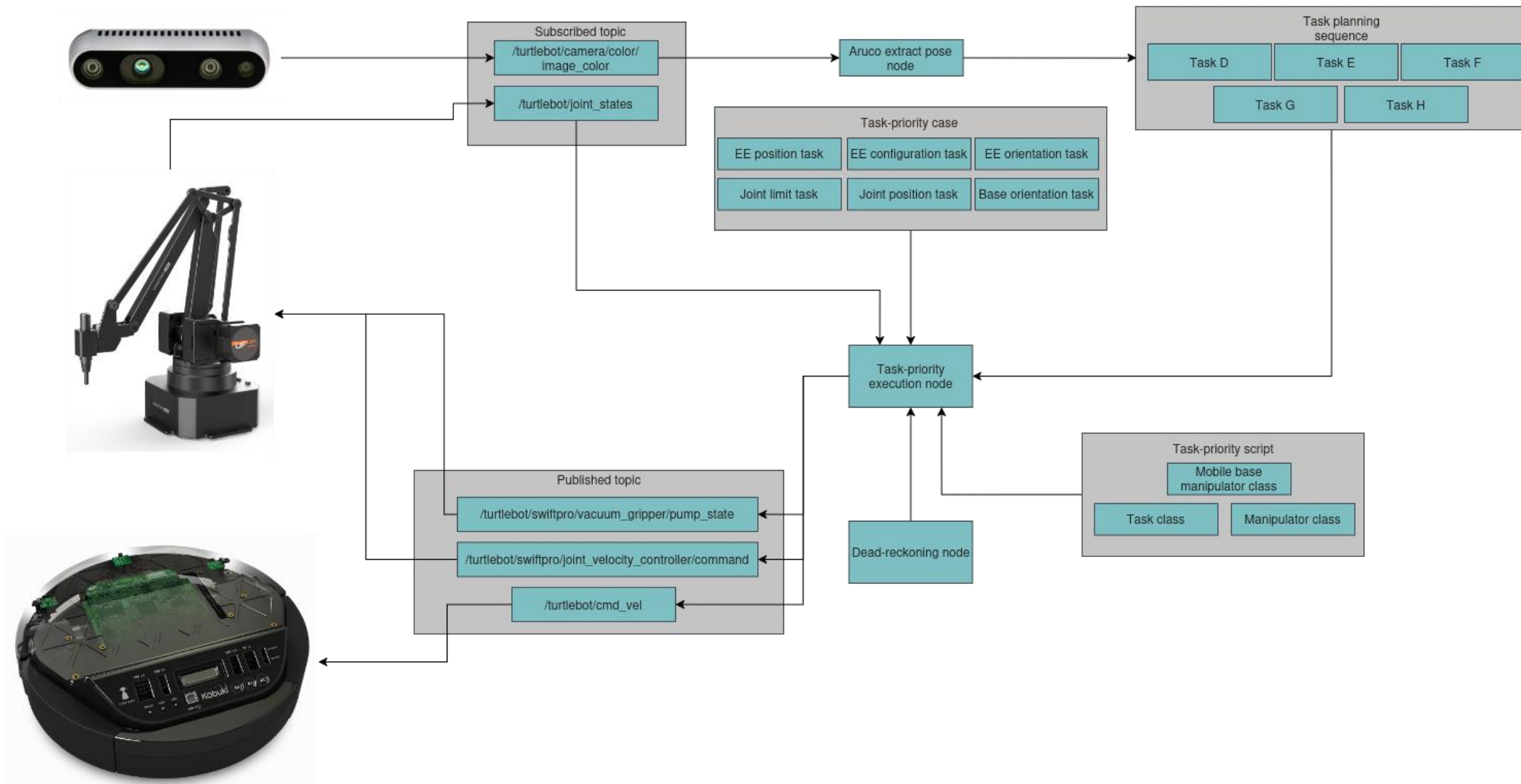
Task Priority Control for Mobile Manipulator

Huu Truong Giang Nguyen
Elchin Aslanli

- 1. Conceptual Design**
- 2. Robot kinematics**
- 3. Task Priority Control**
- 4. Task Planning with Aruco Detection**
- 5. Results**
- 6. Conclusion**

Conceptual Design Phase

Control System Architecture



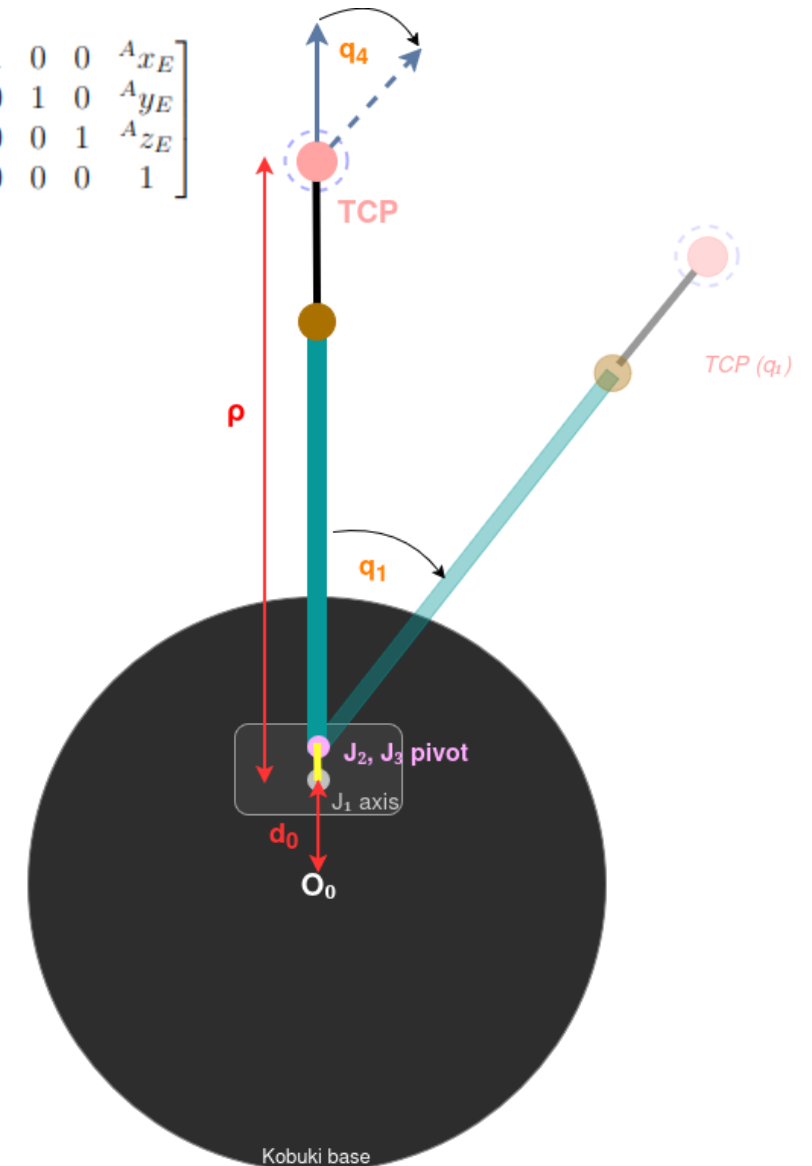
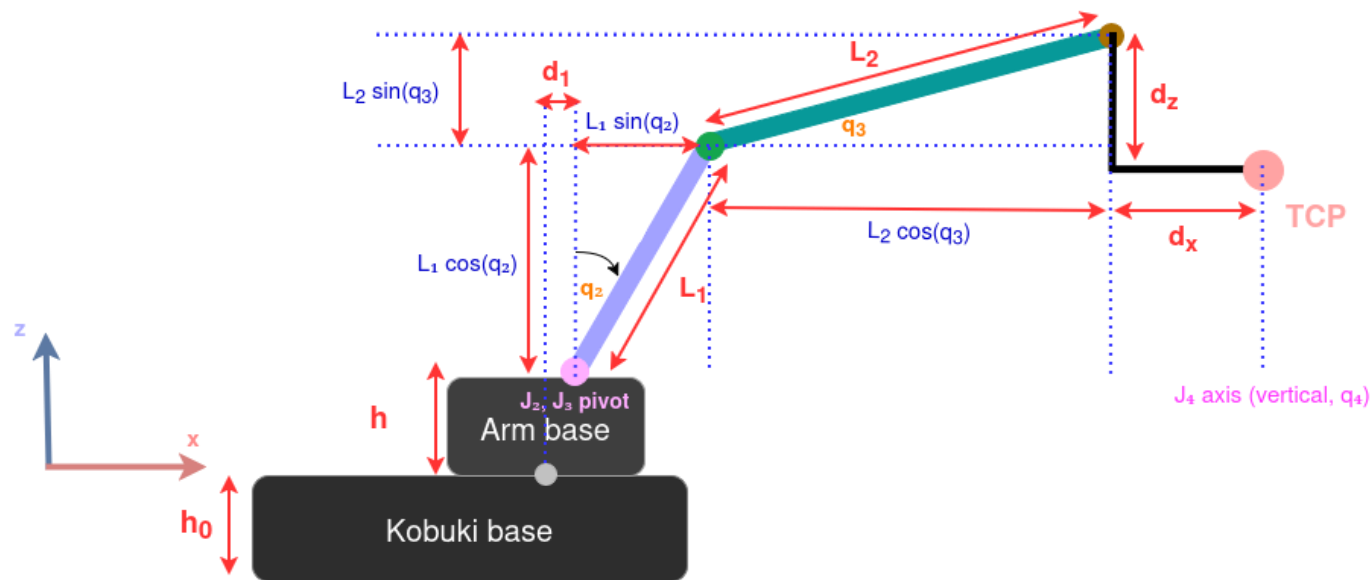
$$\rho = -L_1 \sin q_2 + L_2 \cos q_3 + d_x + d_1$$

$${}^A x_E = \rho \cos q_1$$

$${}^A y_E = \rho \sin q_1$$

$${}^A z_E = -L_1 \cos q_2 - L_2 \sin q_3 - h + d_z$$

$${}^W T_E = {}^W T_B {}^B T_A {}^A T_E = \begin{bmatrix} c\theta & -s\theta & 0 & xR \\ s\theta & c\theta & 0 & yR \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & -1 & 0 & d_0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & -1 & h_0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & {}^A x_E \\ 0 & 1 & 0 & {}^A y_E \\ 0 & 0 & 1 & {}^A z_E \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



Recursive Task Priority with velocity scaling and weighting solution

Recursive Task Priority

Algorithm 2 Recursive Task Priority Redundancy Resolution

Require: A list of tasks

Ensure: Quasi-velocities $\zeta_k \in \mathbb{R}^n$

```
1: Initialise:  $\zeta_0 = 0^n, P_0 = I^{n \times n}$ 
2: for  $i \in 1..k$  do
3:   if  $a_i(q) \neq 0$  then
4:      $\bar{J}_i(q) \leftarrow J_i(q)P_{i-1}$ 
5:      $\zeta_i \leftarrow \zeta_{i-1} + \bar{J}_i^\dagger(q)(a_i(q)\dot{x}_i(q) - J_i(q)\zeta_{i-1})$ 
6:      $P_i \leftarrow P_{i-1} + \bar{J}_i^\dagger(q)\bar{J}_i(q)$ 
7:   else
8:      $\zeta_i \leftarrow \zeta_{i-1}, P_i \leftarrow P_{i-1}$ 
9:   end if
10: end for
11: return  $\zeta_k$ 
```

Velocity Scaling

Algorithm 3 Velocity Scaling

Require: Maximum velocities: $\zeta_{i,max}$ for $i = 1..n(DOF)$

Ensure: Scaled velocities

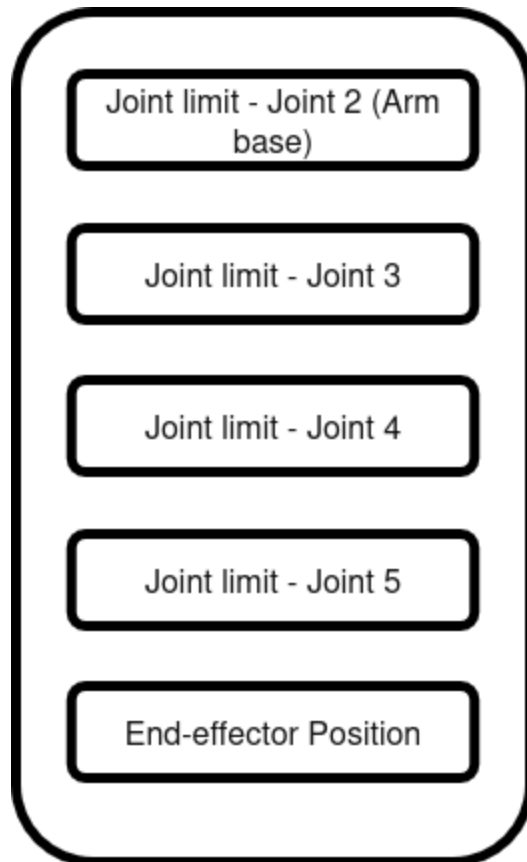
```
1:  $s = \max_{i \in 1..n}(\zeta_i / \zeta_{i,max})$ 
2: if  $s > 1$  then
3:   return  $\zeta / s$ 
4: else
5:   return  $\zeta$ 
6: end if
7: return  $\zeta_k$ 
```

DLS with weighting solution

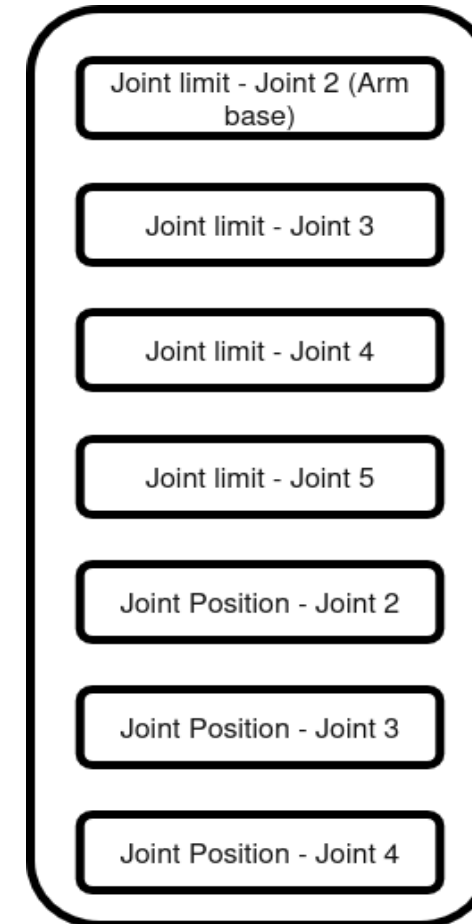
$$\dot{\zeta} = W^{-1}J^T(q)(J(q)W^{-1}J^T(q) + \lambda^2 I)^{-1}\dot{x}_E$$

Task priority:

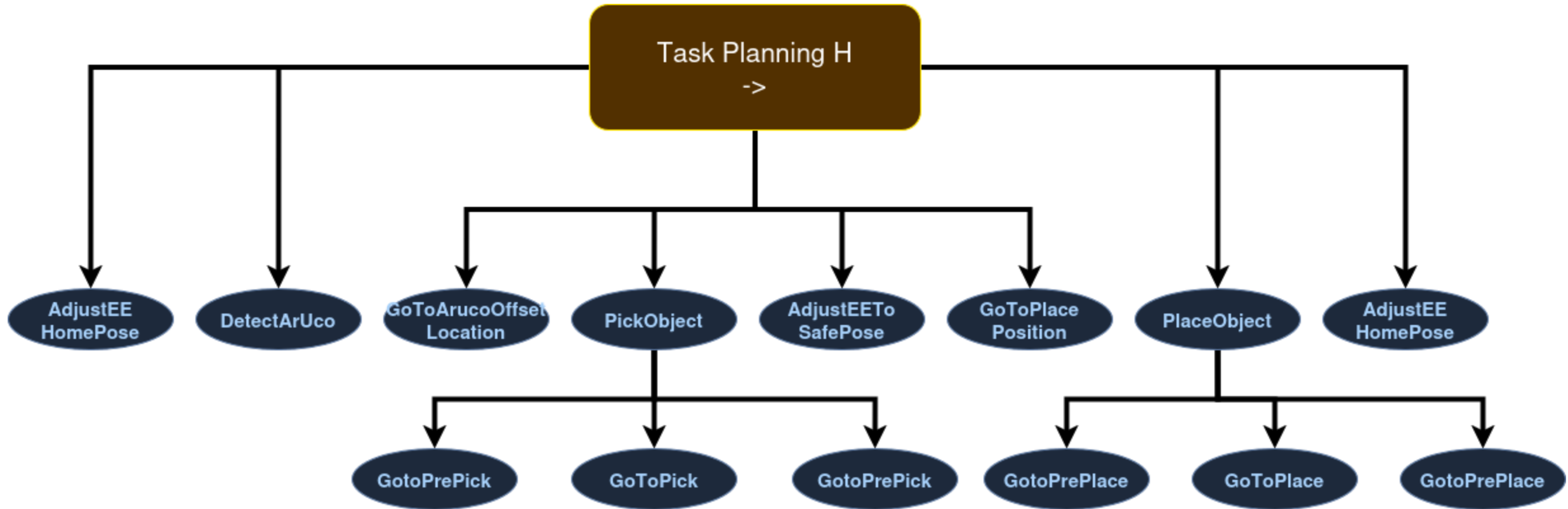
Move arm to desired position



Adjust arm position (Home or Safe position)

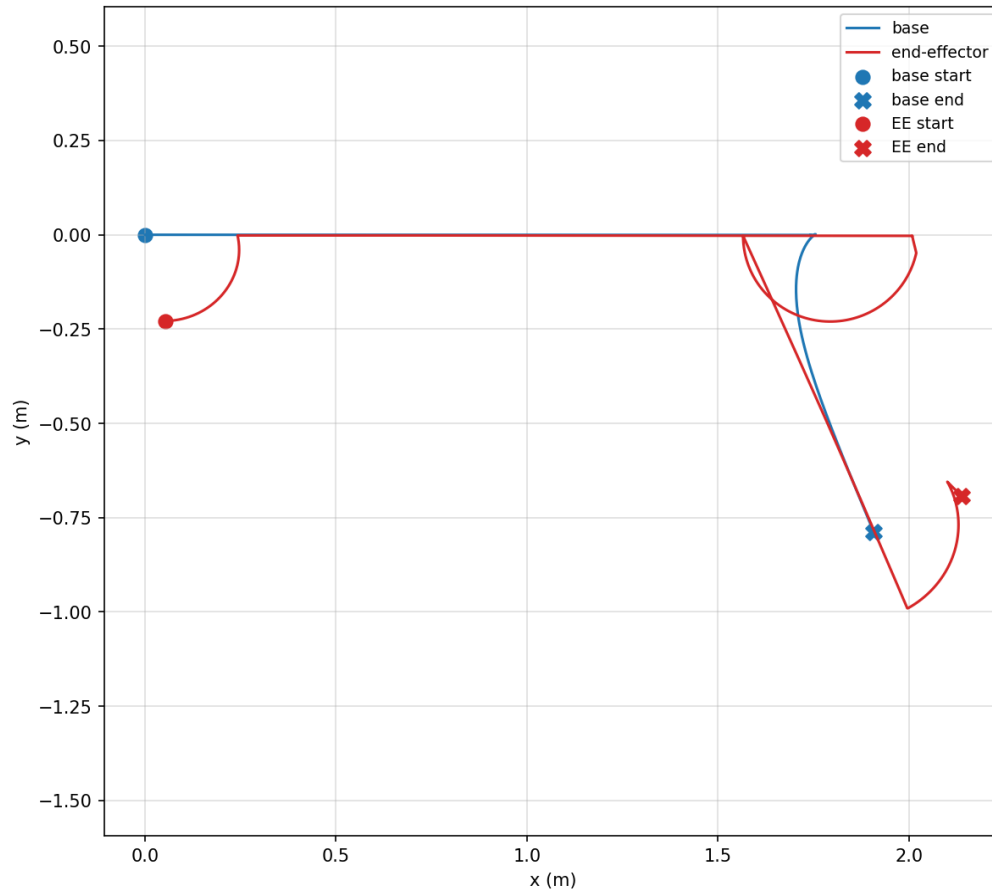


Task Planning with Aruco Detection



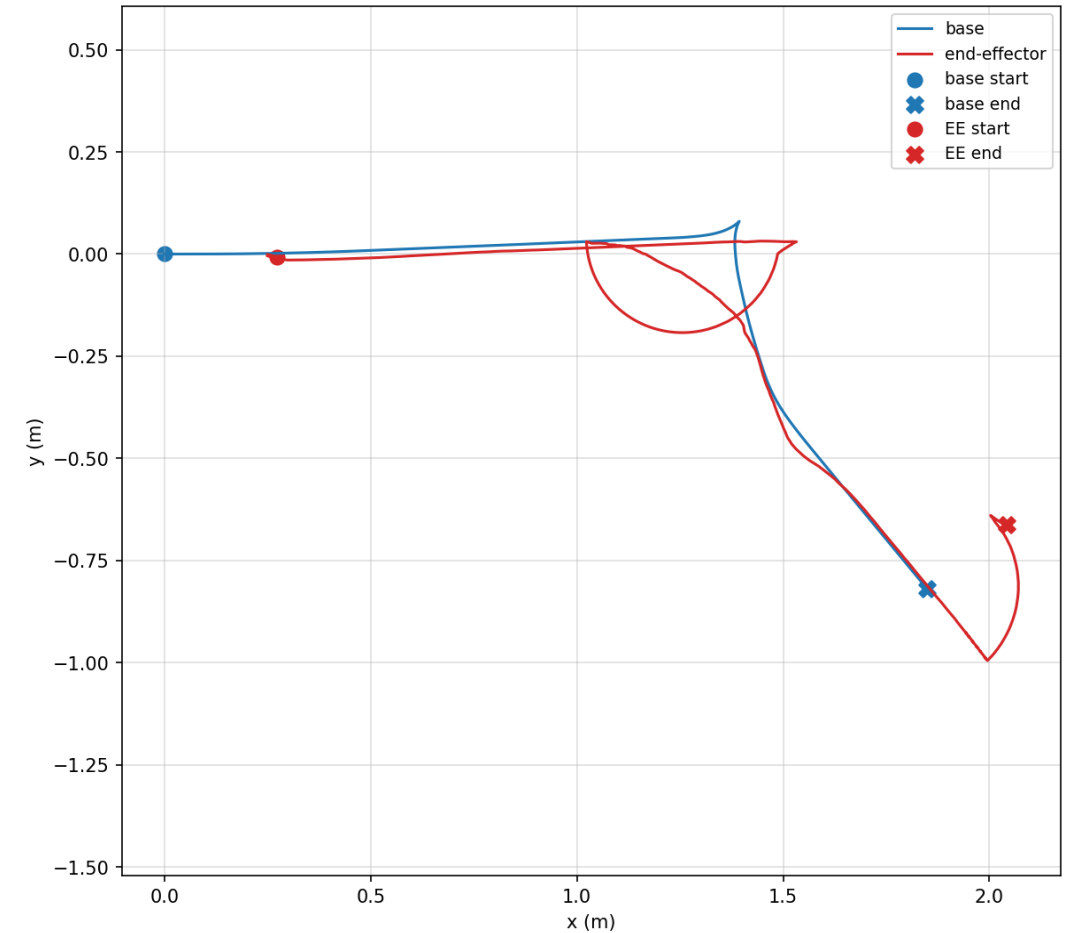
Simulation

Base and End-Effector Path (XY plane)



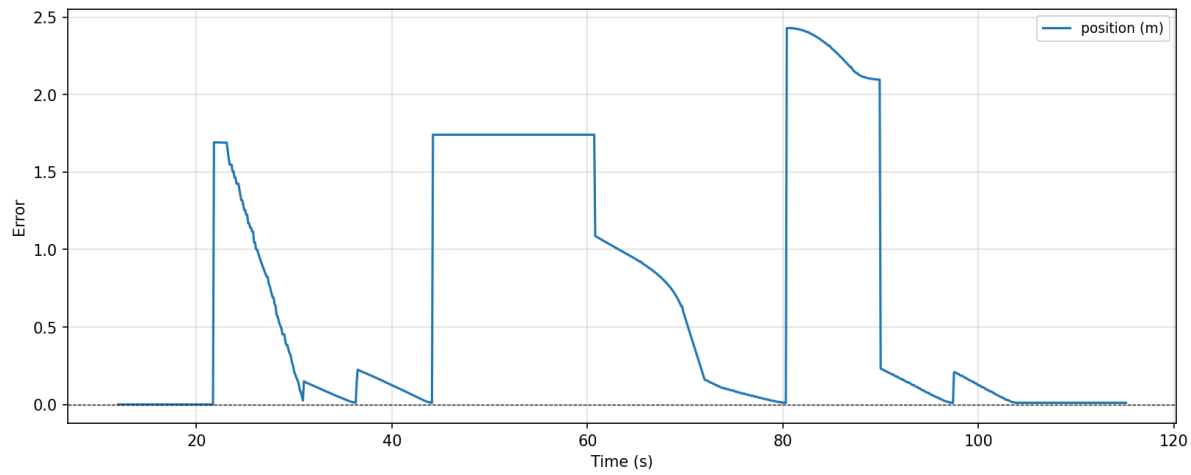
Real

Base and End-Effector Path (XY plane)



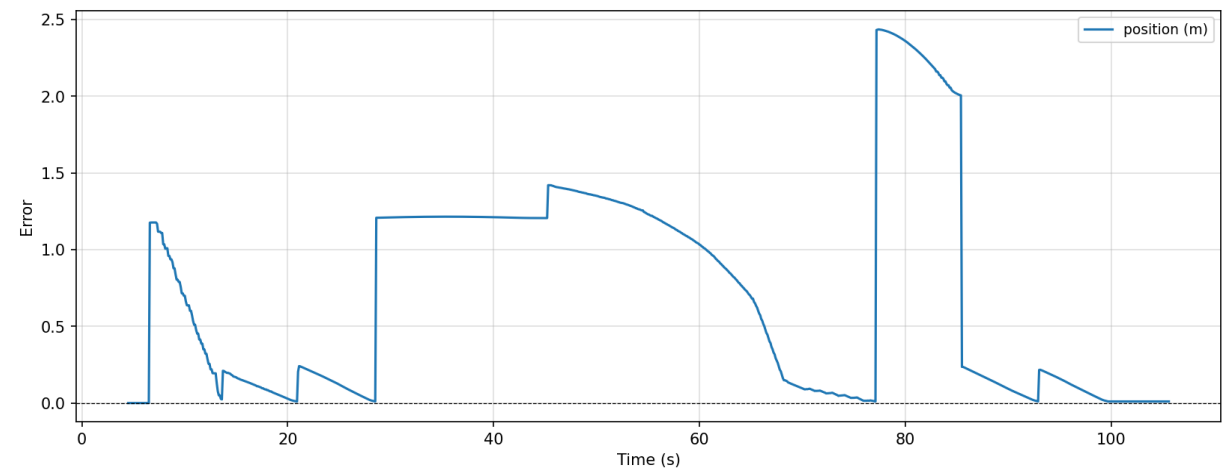
Simulation

End-Effector Position Error (desired – actual)



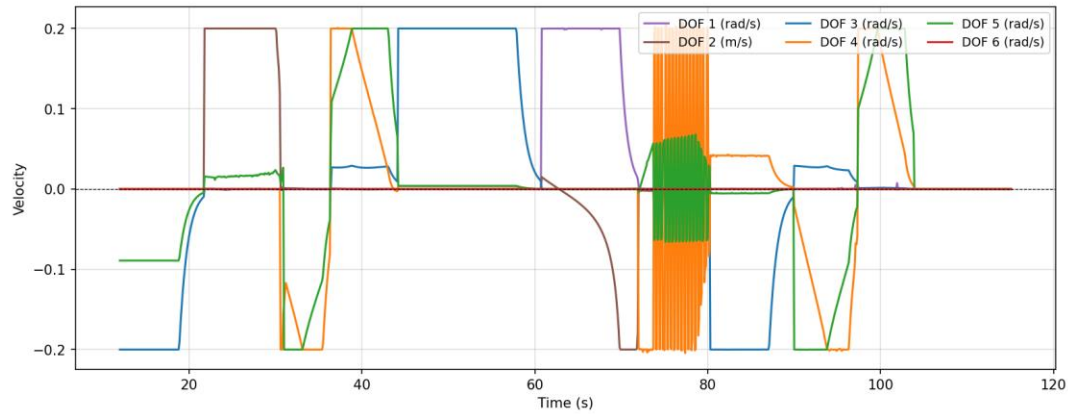
Real

End-Effector Position Error (desired – actual)

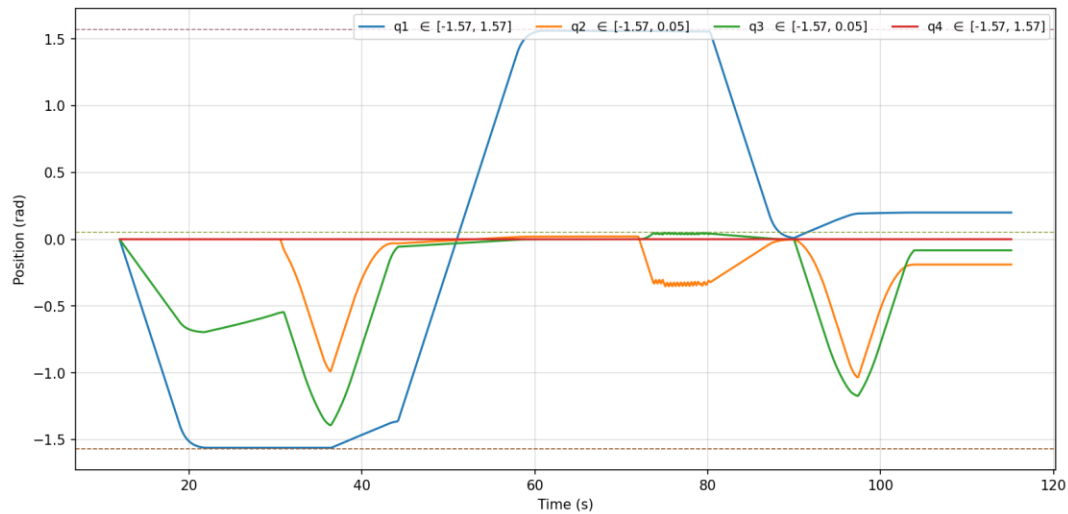


Simulation

DOF Velocities (base + arm joints)

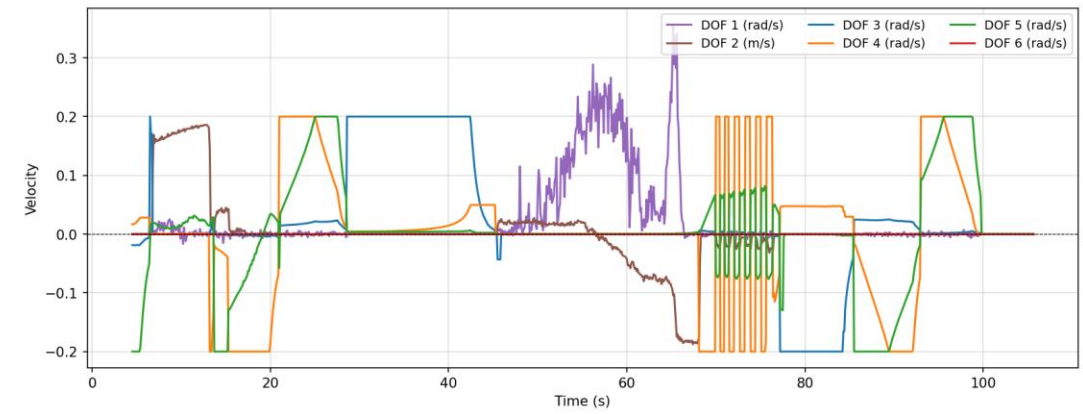


Joint Positions with Limits

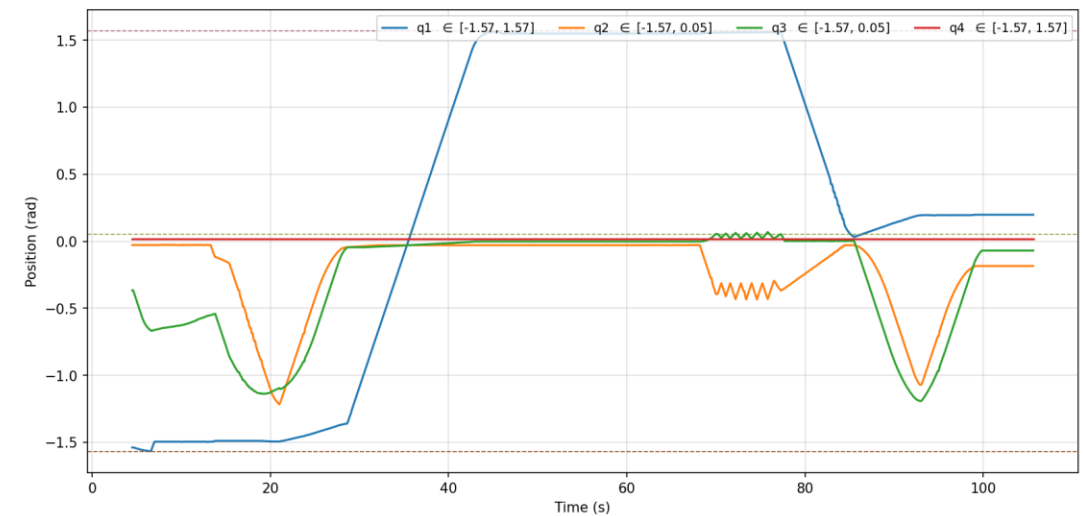


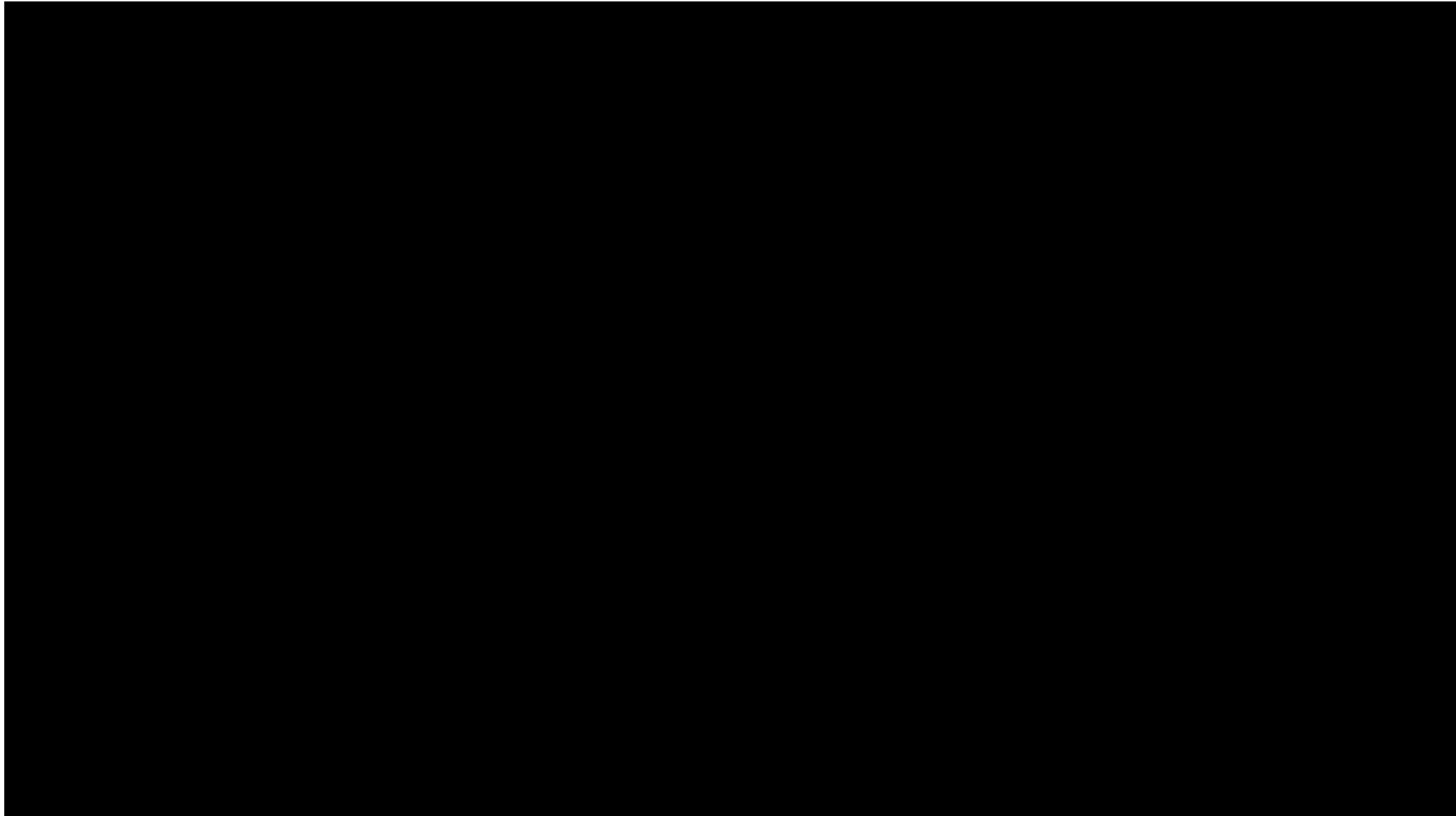
Real

DOF Velocities (base + arm joints)



Joint Positions with Limits





Conclusion:

The task priority works successful in simulation and real robot with practical implementation: velocity scaling and weighting solution

Completing implement behavior tree with task priority and aruco extraction to finish task H (Pick and Place).

Future work:

Extend our work with more flexible situation such as robot miss the aruco and it will automatically go to the behavior finding the aruco back to pick it.